Constructor University Bremen

Fall Semester 2025

Robotics and Intelligent Systems Lab

Michael Görner

Assignment 04

Ali Ahmed Mohamed Amaal Sayed Labib Ibrahim

Yassein Ahmed Mahmoud

Yassin Baher Youssef

ROBOT 1: UR5e

Figure 1 RViz overview of the UR5e robot and camera feed:

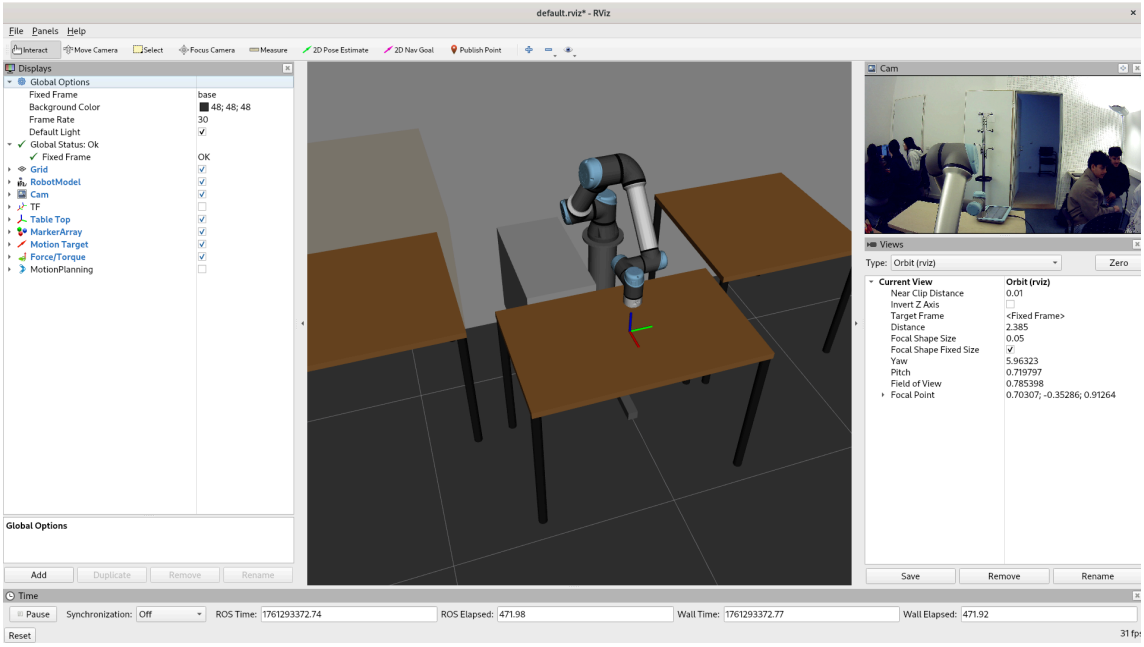
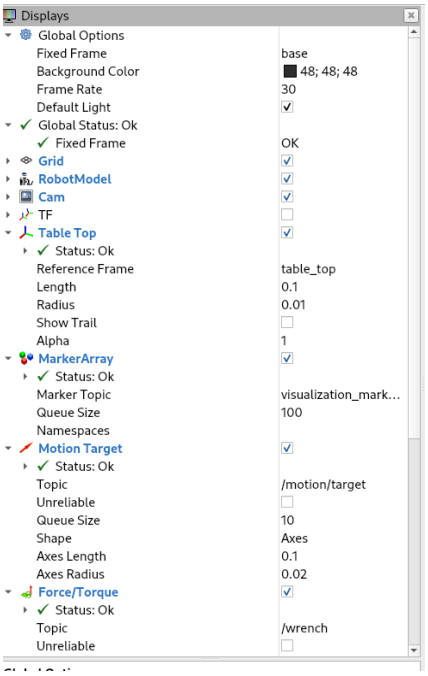


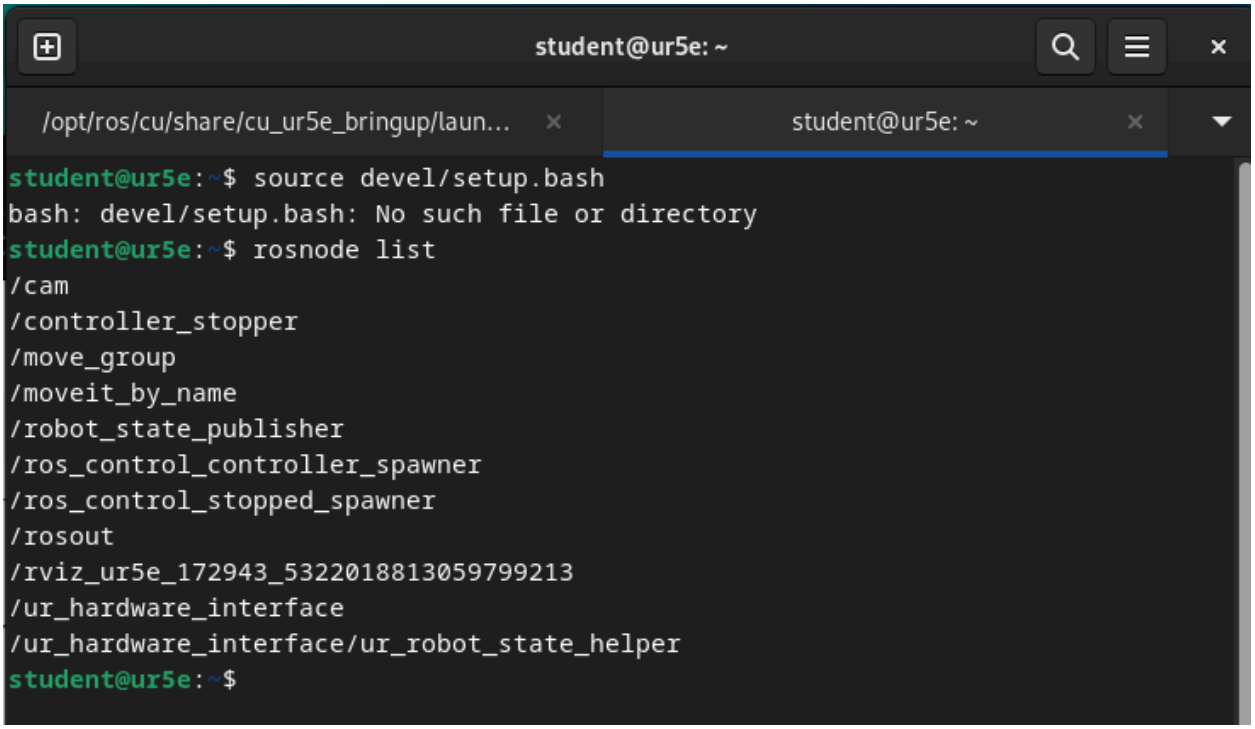
Figure 2 Displays panel showing all configured topics and statuses:



In RViz, several displays were configured to visualize the UR5e robot and its environment. The Grid provided a ground reference for orientation, while the RobotModel and TF displays showed the robot's structure and coordinate frames. The Cam and Table Top displays added visual context of the workspace, and the MarkerArray, Motion Target, and Force/Torque displays helped visualize object markers, target poses, and applied forces. The MotionPlanning display

was disabled in this setup but is normally used for interactive path planning. Together, these displays gave a clear and complete view of the robot's configuration, motion, and environment.

Figure 3 ROS nodes running for UR5e (rosnode list output):



The three main ROS nodes active in the UR5e bring up were:

- /robot_state_publisher: Publishes all link transforms (TF tree) based on /joint_states and the URDF model in /robot_description.
- /ur_hardware_interface: Communicates directly with the physical UR5e controller, sending joint commands and receiving state feedback.
- /move_group: Core MoveIt node responsible for motion planning, inverse kinematics, collision checking, and trajectory execution.

Figure 4 UR5e configuration files inside the config directory:

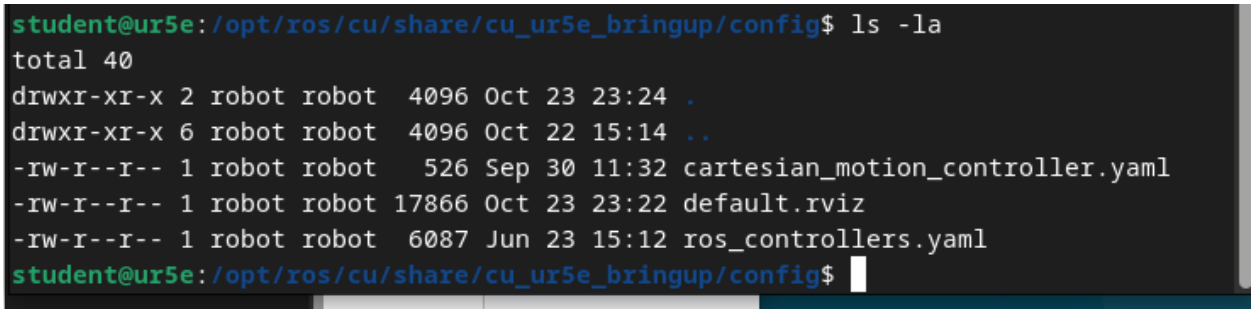
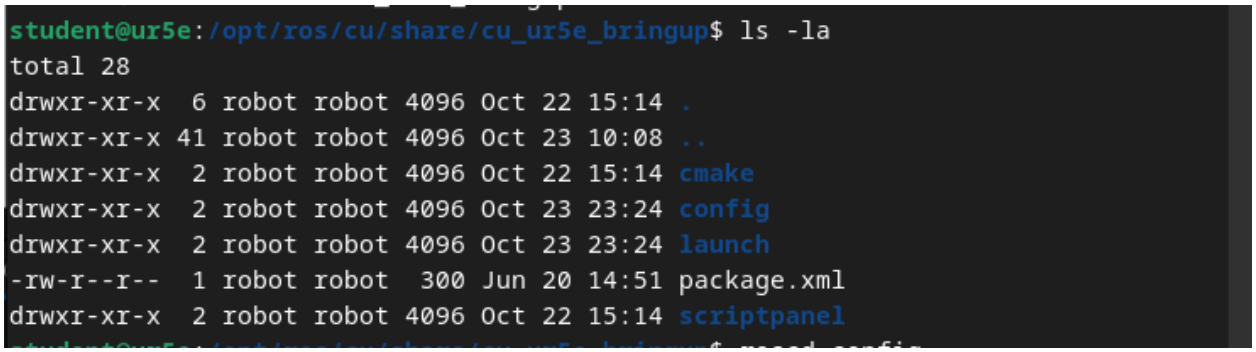


Figure 5 UR5e bring-up package structure:

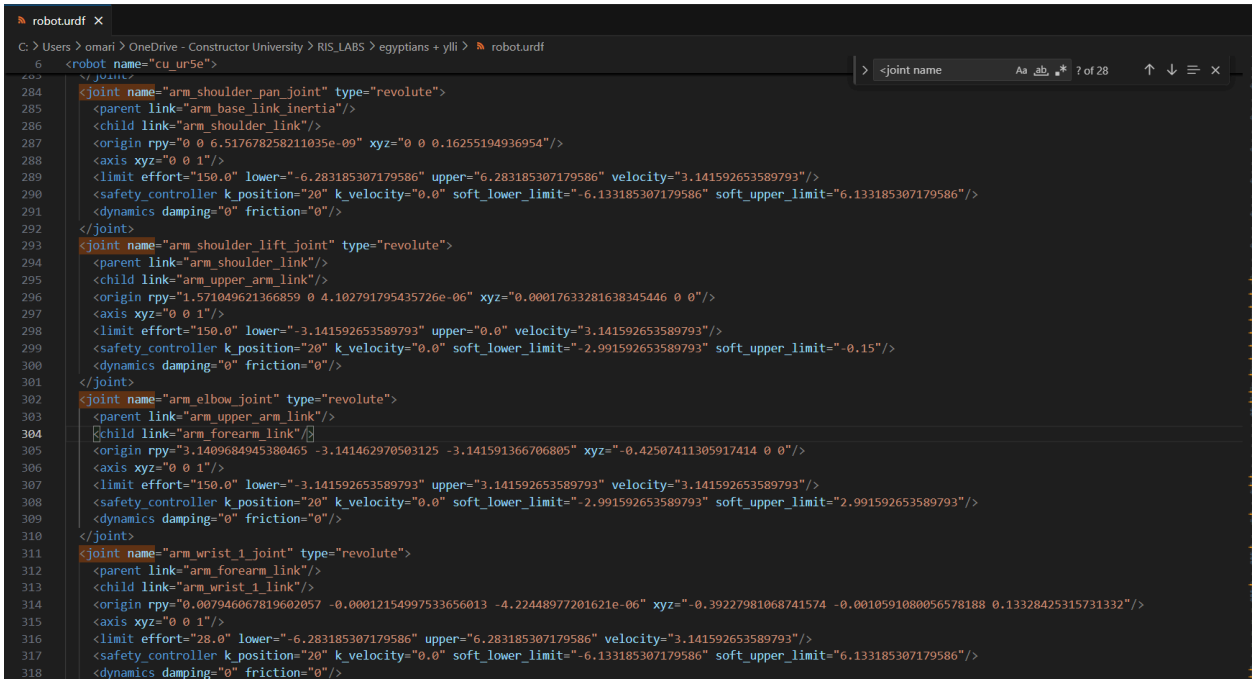


The UR5e bring-up directory contains several important subfolders and files that are loaded when running roslaunch cu_ur5e_bringup hardware.launch., including:

- config/: contains setup files such as ros_controllers.yaml (controller configuration), cartesian_motion_controller.yaml (motion control setup), and default.rviz.
- launch/: includes the main launch files that start the robot's control, MoveIt planning, and visualization nodes.
- cmake and scriptpanel: provide supporting scripts and build configuration for running the package.

After running the hardware launch file, we saw that the UR5e model appeared in RViz, meaning that the URDF was successfully loaded into /robot_description The controllers also started correctly, and the MoveIt interface became active, allowing us to visualize and test the robot's movements directly from the planning scene.

Figure 6 UR5e URDF file showing joint limit parameters:



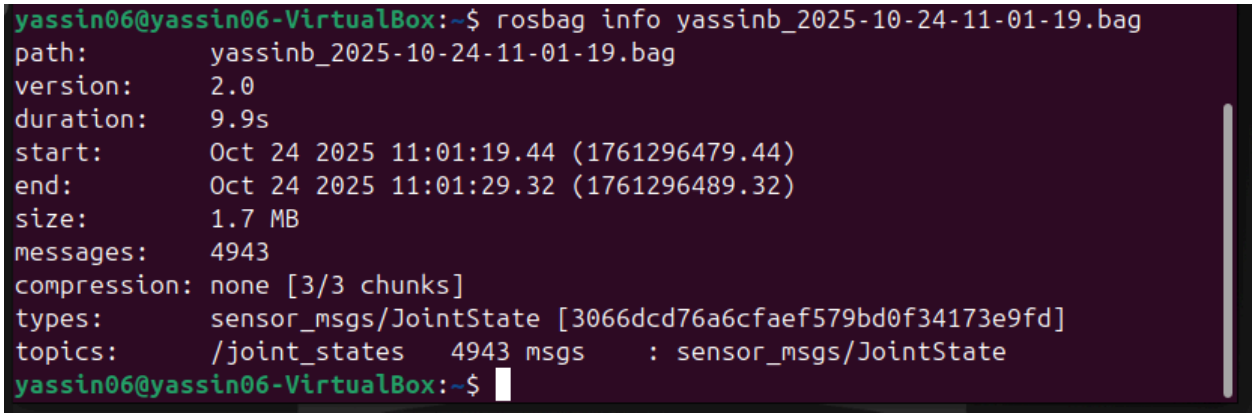
The robot's structure and kinematics are defined in its URDF, which is automatically loaded into the ROS parameter /robot_description when launching the UR5e.
The file specifies all links, joints, and their motion limits.

The UR5e includes six revolute joints with limits defined as:

- Shoulder pan: ±6.283 rad, effort 150 N·m
- Shoulder lift: −3.142 to 0 rad, effort 150 N·m
- Elbow: ±3.142 rad, effort 150 N·m
- Wrist 1–3: ±6.283 rad, effort 28 N·m

These limits set the allowed range, torque, and speed of each joint, ensuring safe and realistic robot motion in both RViz and physical operation.

Figure 7 Verification of recorded /joint_states data using rosbag info:

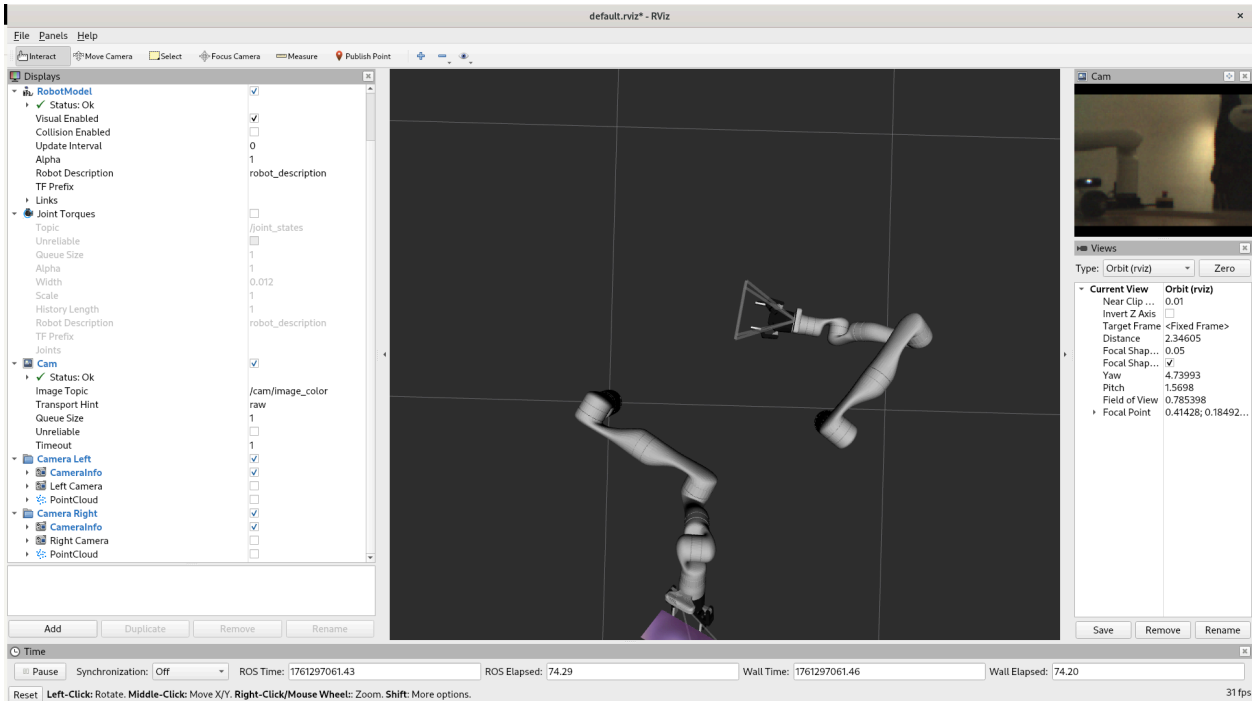


Each group member recorded a short rosbag of the /joint_states topic by moving the UR5e arm in freemode to write the first letter of their name in the air. The motion was visualized in RViz using the TF Trajectory plugin and checked afterward with the rosbag info command. Figure 7 shows that the file yassinb_2025-10-24-11-01-19.bag contains 4943 messages over a 9.9-second recording, confirming the data was successfully captured.

ROBOT 2: Kinova

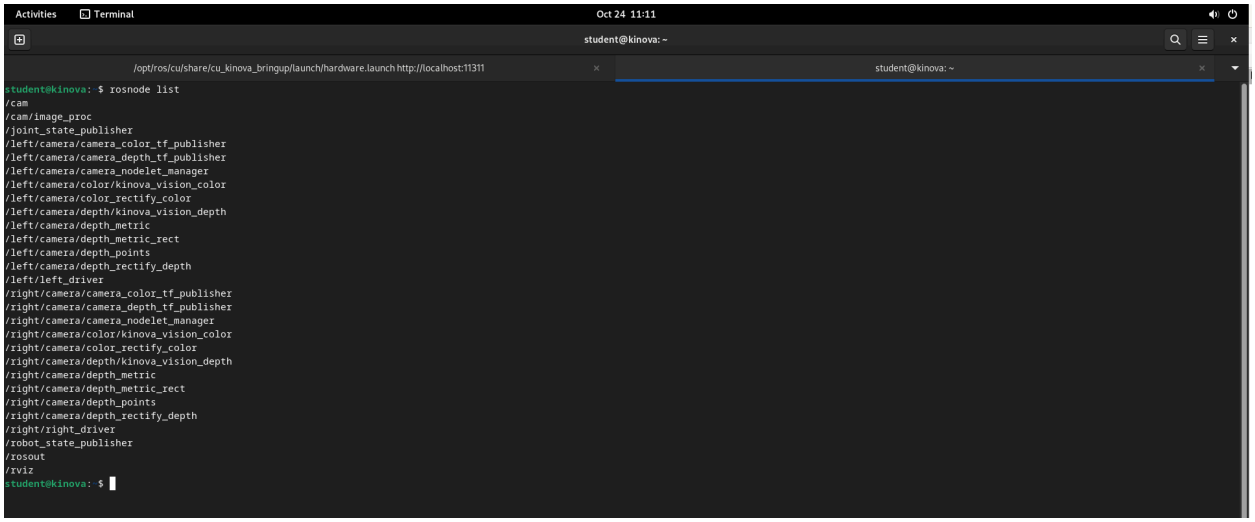
Task 1:

Figure 8 RViz overview of the Kinova robot and configured displays:



In RViz, the configuration for the Kinova robot included several useful displays. The Grid provided a ground reference to help visualize the robot's position in space. The RobotModel showed the robot's full structure based on its URDF, while the TF display visualized the coordinate frames of each joint and link. The Cam display streamed the live camera view from the robot's workspace. Other displays such as MarkerArray and Motion Target were available but not active in this setup. Overall, these displays worked together to give a clear visualization of the Kinova Gen3 robot's configuration, movements, and environment.

Figure 9 ROS nodes running for the Kinova (rosnode list output):

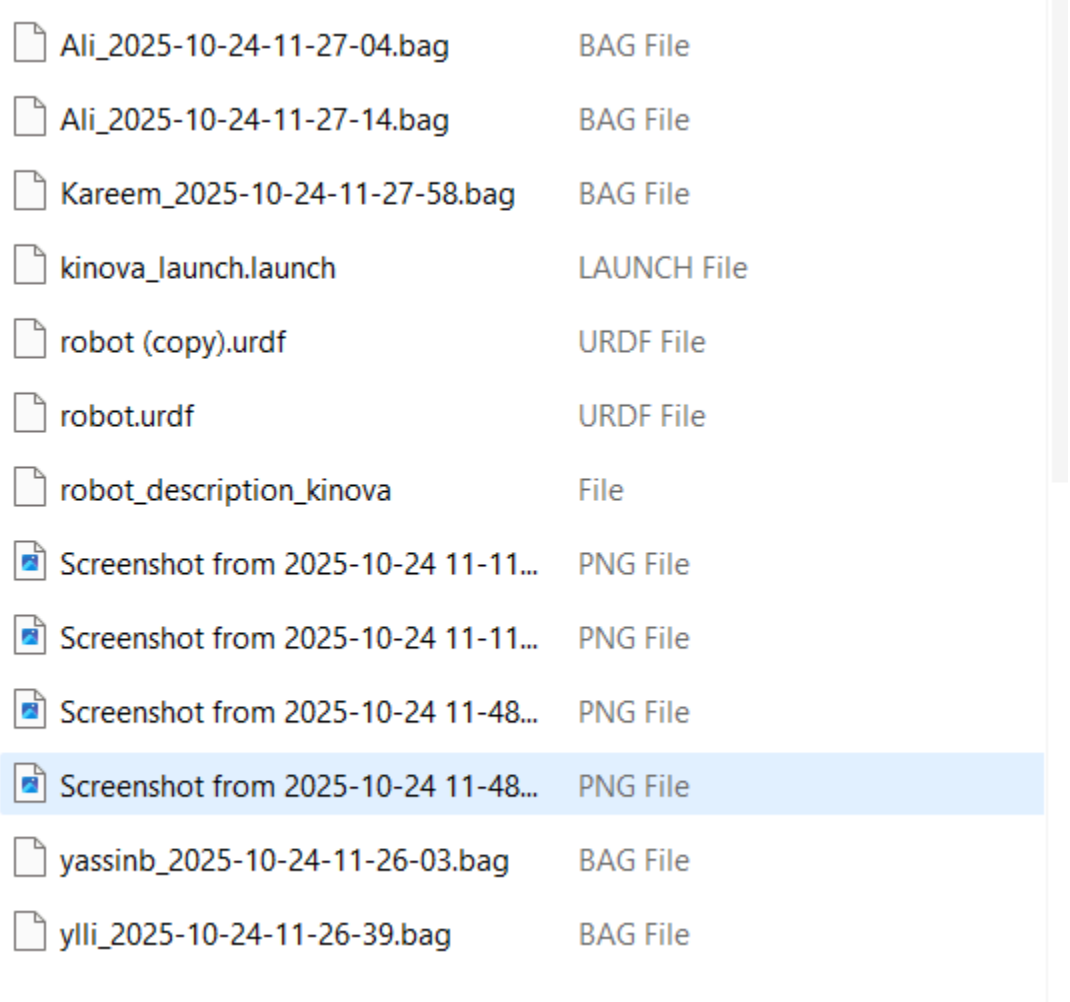


The rosnode list output displayed all active nodes running during the Kinova bring-up.

Three key nodes were selected and explained below:

- /robot_state_publisher: Publishes all link transforms (TF tree) based on /joint_states and the URDF loaded into /robot_description, allowing RViz to visualize the robot's configuration in real time.
- /kinova_driver: Manages the communication between ROS and the Kinova hardware, sending motion commands to the robot's actuators and receiving feedback from sensors and encoders.
- /move_group: Provides high-level motion planning, inverse kinematics, and trajectory execution through the MoveIt framework, enabling smooth and collision-free arm movement

Figure 10 Kinova bring-up package structure showing launch and URDF files:

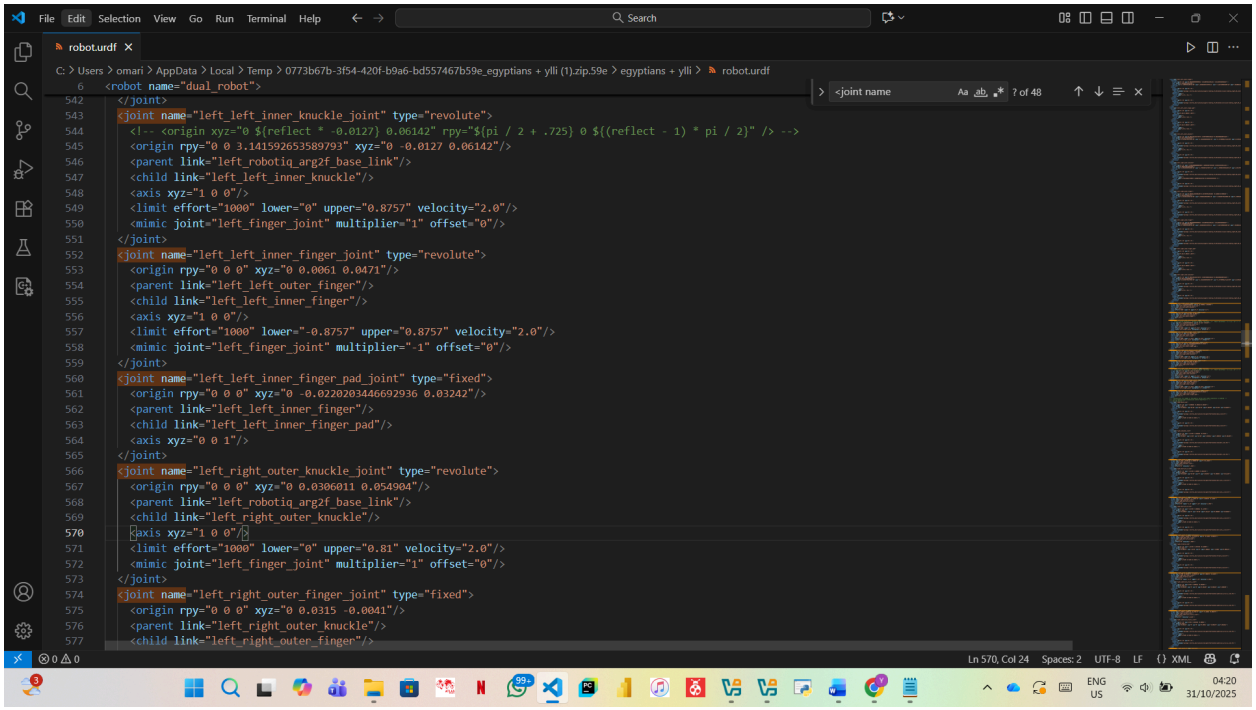


The Kinova bring-up folder includes key files like kinova_launch.launch, robot.urdf, and configuration folders that define how the robot is set up in ROS. When running the launch file, several components are started together to bring the robot online:

- Robot Description Loader: Loads the robot's URDF model into /robot_description, allowing visualization and kinematic data in RViz and MoveIt.
- Controller Spawner: Loads the controller settings from the configuration files to manage the robot's torque and position control.
- Hardware Driver: Launches the kinova_driver node, which communicates with the Kinova arm and gripper.
- MoveIt Bring-Up: Starts /move_group for motion planning, inverse kinematics, and trajectory execution.
- RViz Configuration: Opens a predefined RViz setup to visualize the robot, its coordinate frames, and camera view.

This setup ensures that all main modules for visualization, control, and planning work together smoothly when operating the Kinova.

Figure 11 Kinova URDF file showing joint limit parameters:



The Kinova robot's structure and kinematics are defined in its URDF file, which is automatically loaded into the ROS parameter /robot_description when launching the robot. The file specifies all the robot's links, joints, and their motion limits. The Kinova includes seven revolute joints for the arm and additional joints for the Robotiq gripper fingers. Each joint has its own limits for effort, range, and speed. For example, the gripper joints have limits defined as:

- Lower: –0.8757
- Upper: 0.8757
- Velocity: 2.0 rad/s

These parameters control how far and how fast each finger can move, ensuring smooth and safe operation during grasping.

Figure 12 Verification of recorded /joint_states data using rosbag info:

```
yassin06@yassin06-VirtualBox:~$ rosbag info yassinb_2025-10-24-11-26-03.bag
path:        yassinb_2025-10-24-11-26-03.bag
version:     2.0
duration:    10.0s
start:       Oct 24 2025 11:26:03.97 (1761297963.97)
end:         Oct 24 2025 11:26:13.00 (1761297974.00)
size:        528.1 KB
messages:    402
compression: none [1/1 chunks]
types:       sensor_msgs/JointState [3066dcd76a6cfaef579bd0f34173e9fd]
topics:      /joint_states   402 msgs    : sensor_msgs/JointState
yassin06@yassin06-VirtualBox:~$
```

Each group member recorded a rosbag of the /joint_states topic by moving the Kinova arm in freedrive mode to write the first letter of their name in the air. The motion was visualized in RViz using the TFTrajectory display to show the path of the movement. After recording, the bag files were checked with the rosbag info command. Figure 12 shows that the file yassinb_2025-10-24-11-26-03.bag contains 402 messages over a 10-second duration, confirming that the recording was successful.

Task 2:

Figure 13 Gripper open (position = 0.0):

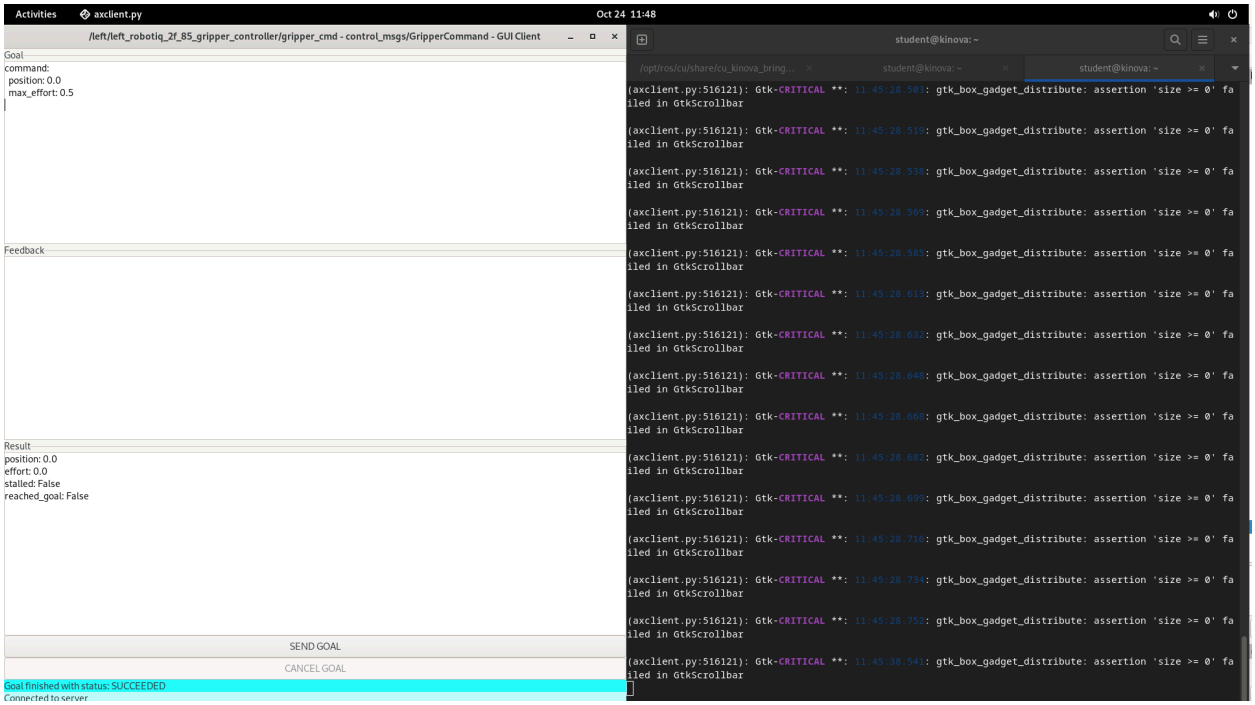


Figure 14 Gripper closed (position = 0.8):

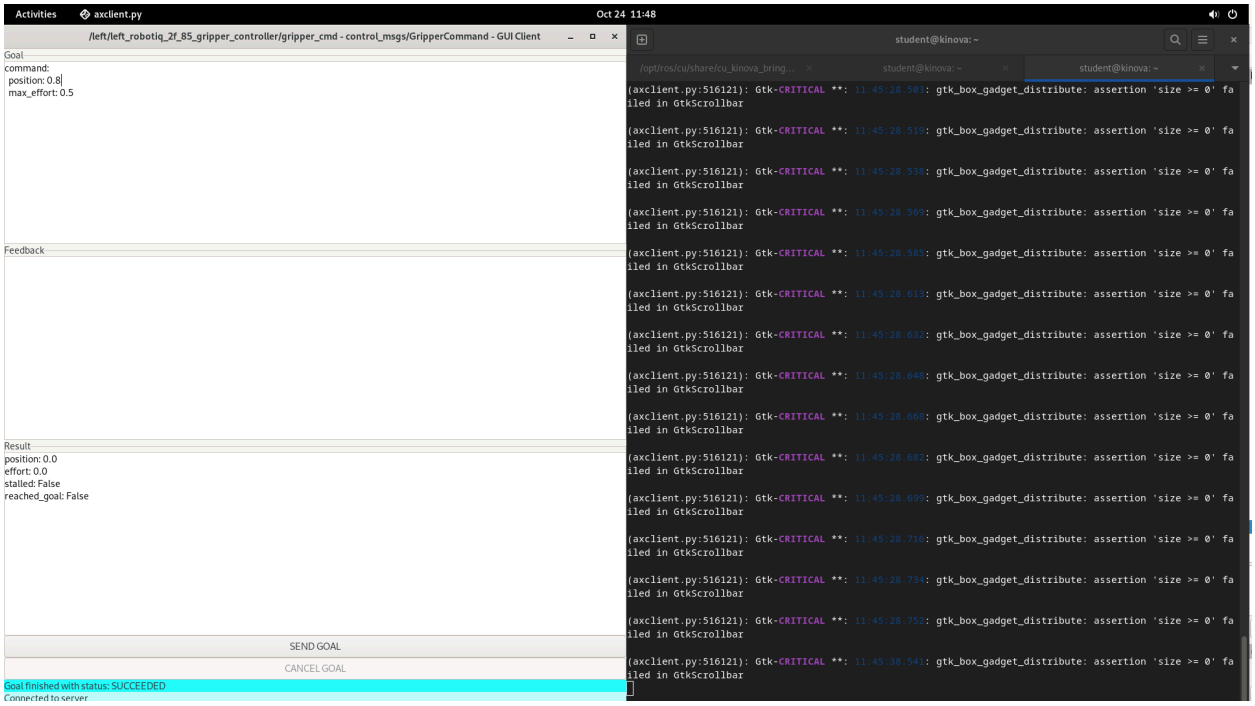


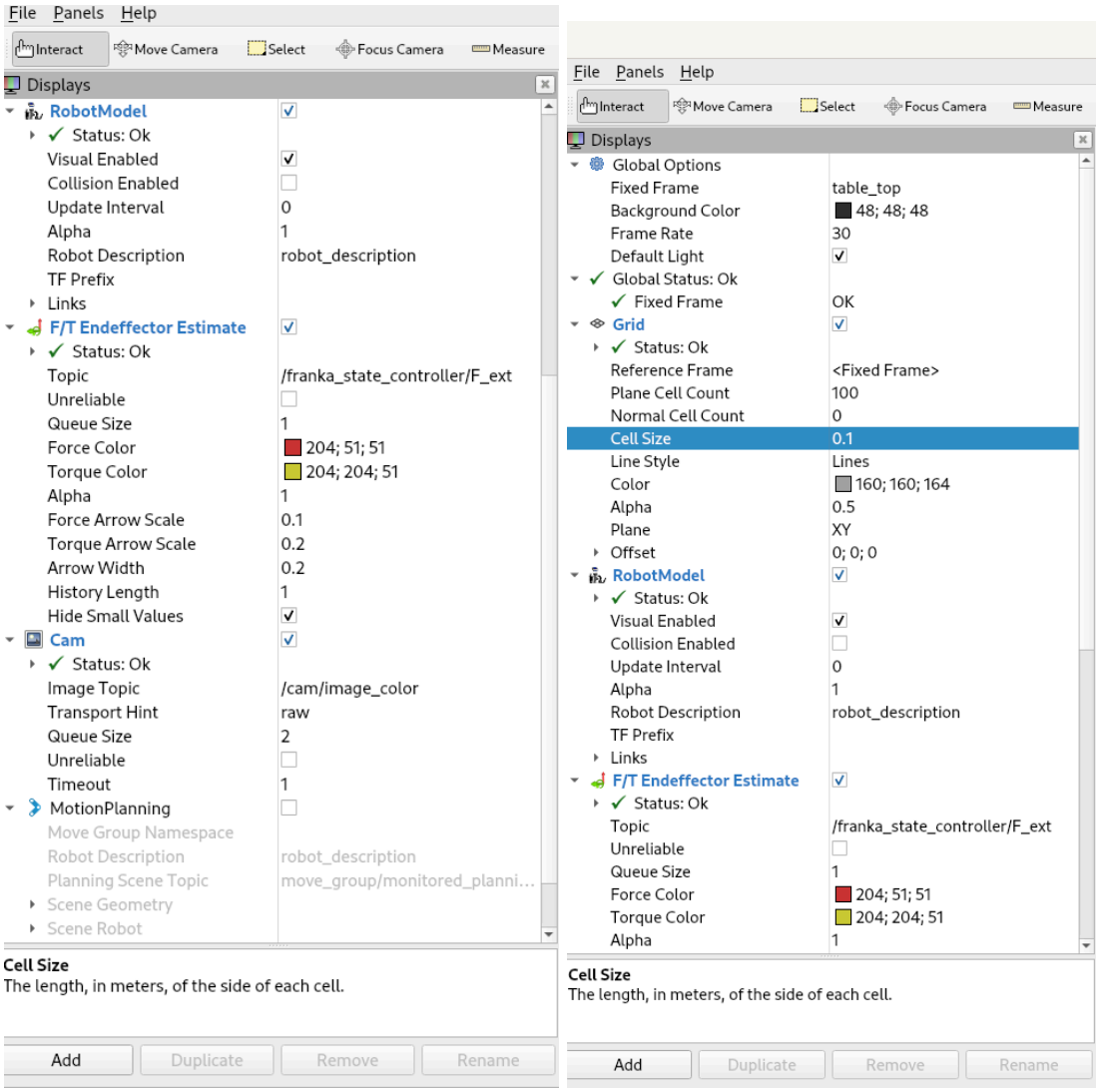
Figure 13 Gripper action client GUI before sending command (open position)

Figure 14 Gripper action client GUI after sending command (closed position)

The Kinova robot uses a Robotiq 2F-85 gripper, which can be controlled through a ROS Action interface. Using the axclient.py tool, we sent different commands to open and close the gripper by setting values for position and max effort. When the position was set to 0.0, the gripper opened fully, and when set to 0.8, it closed. The max effort value was set to 0.5 to control how tightly it grips. After sending each command, the feedback window showed that the goal was completed successfully with the message “Goal finished with status: SUCCEEDED.” This confirmed that the gripper responded correctly and that the ROS Action system worked as expected.

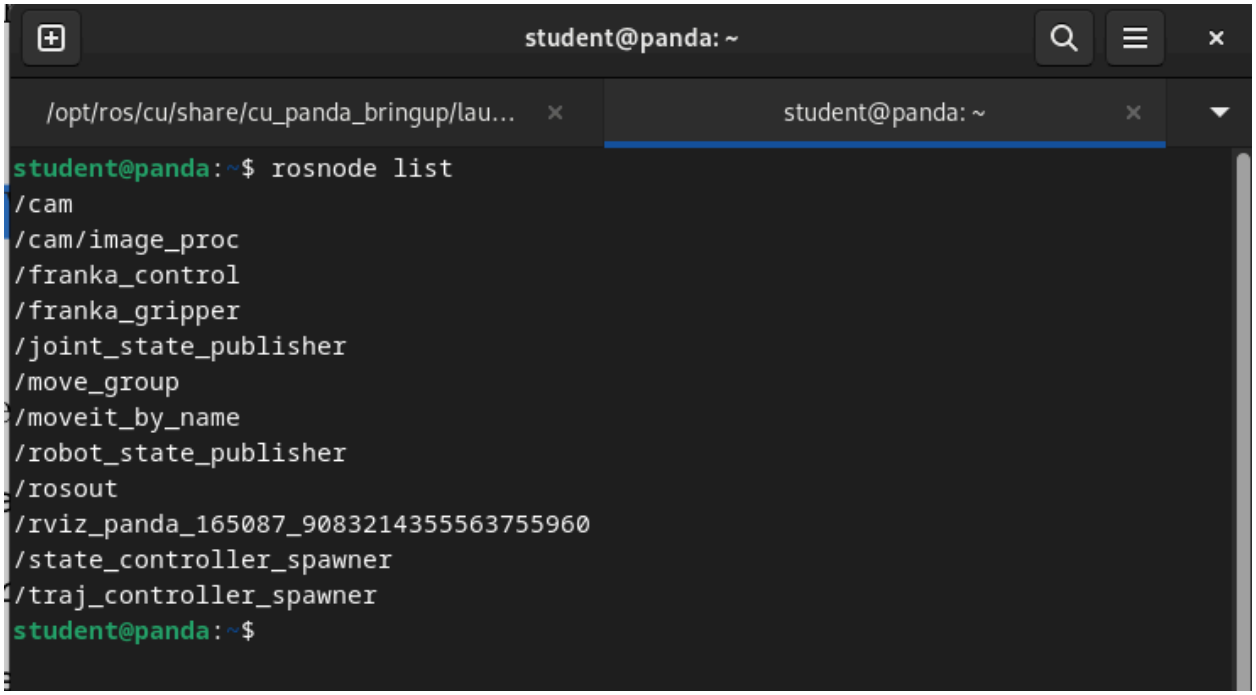
ROBOT 3: Panda

Figure 15 RViz overview of the Panda robot and configured displays:



In RViz, the Panda robot's setup included several displays. The Grid gave a ground reference for positioning, and the RobotModel showed the full 3D model of the robot. The F/T Endeffector Estimate displayed the forces and torques at the robot's end-effector, while the Cam display showed the live camera feed of the workspace. The MotionPlanning display was listed but not enabled, as it's mainly used for path planning and collision checks. Together, these displays helped visualize the Panda robot's structure, camera view, and interaction in the environment.

Figure 16 ROS nodes running for the Panda (rosnode list output):

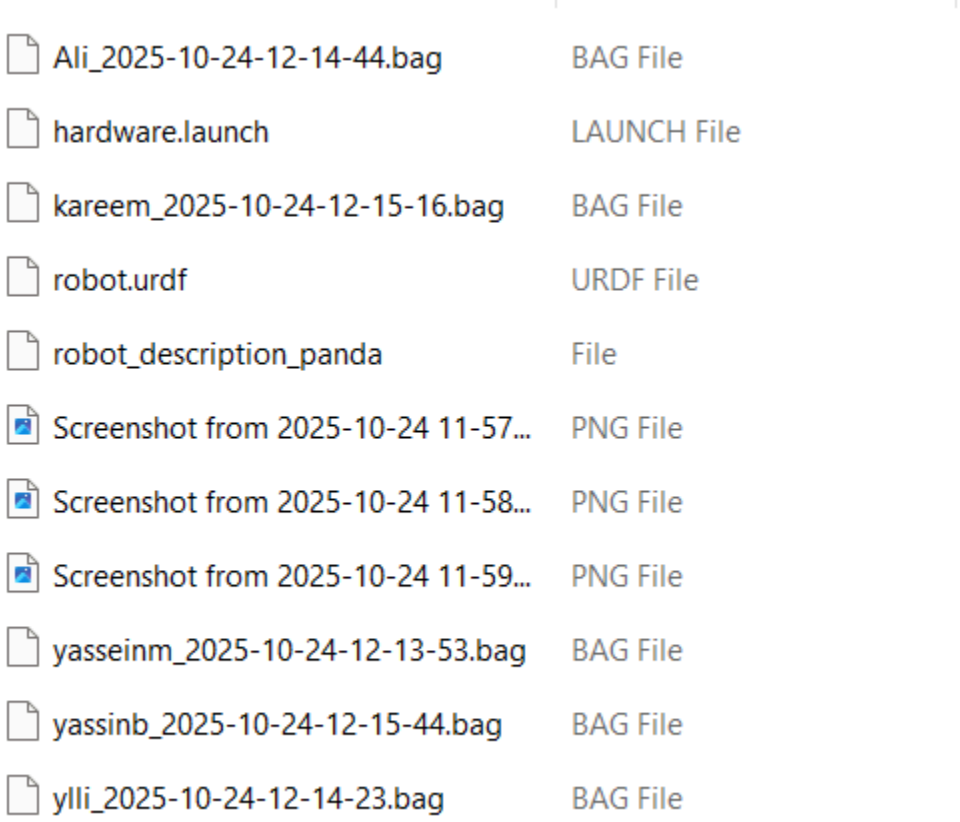


Three key ROS nodes were selected and explained below:

- /franka_control – Handles low-level control of the Panda arm, managing torque and impedance for smooth and precise motion.
- /franka_gripper – Controls the gripper's open and close actions and monitors its current state.
- /move_group – Provides motion planning and inverse kinematics via the MoveIt framework, allowing for path execution and collision avoidance.

These nodes together form the core communication and control system of the Panda robot, enabling stable arm and gripper operation during experiments.

Figure 17 Panda bring-up package structure showing launch and URDF files:

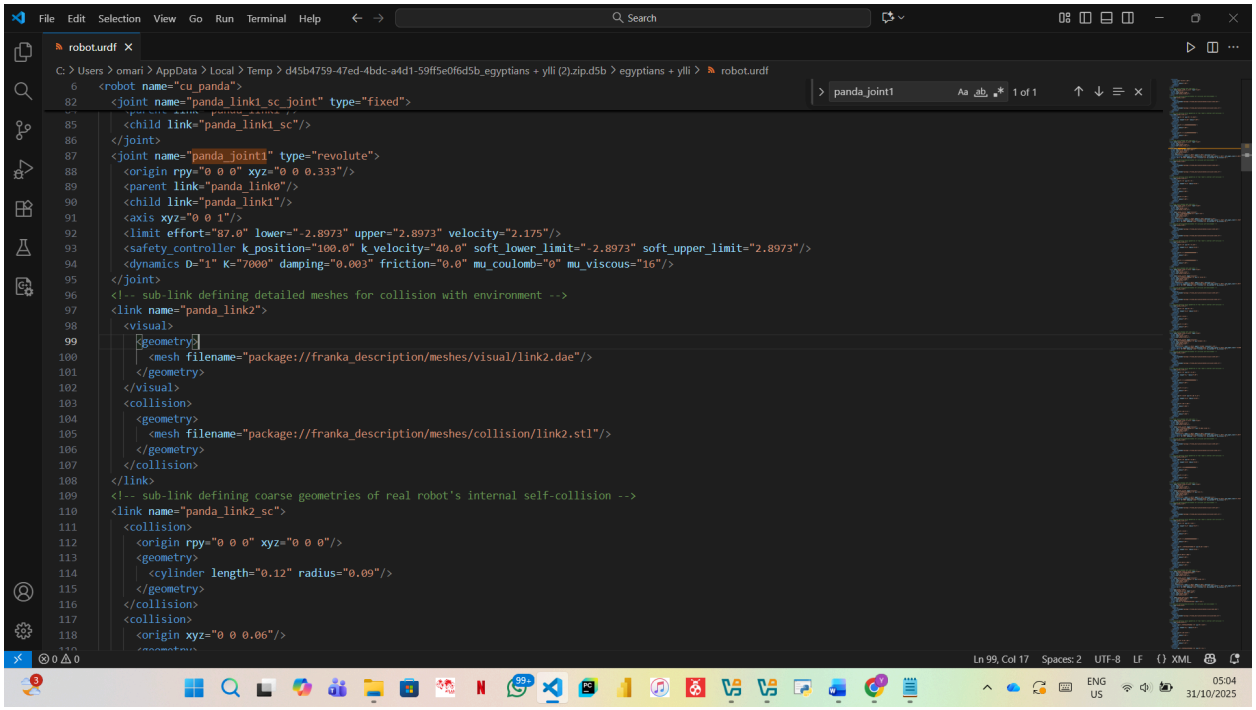


The Panda robot uses the hardware.launch file to start all the main parts it needs to work. This file loads the robot's URDF model and starts the nodes for control, planning, and visualization.The main parts it launches are:

- Robot Description Loader: Loads the robot model into /robot_description for RViz and MoveIt.
- Controller Spawner: Starts the controllers that move the robot's joints.
- Franka Control Node: Handles smooth and precise movement of the arm.
- MoveIt Bring-Up: Enables motion planning so the robot can move safely.
- RViz Configuration: Opens the 3D view of the robot and its camera feed.

This setup makes sure the Panda robot is fully ready for control and visualization in ROS.

Figure 18 Panda URDF file showing joint limit parameters:

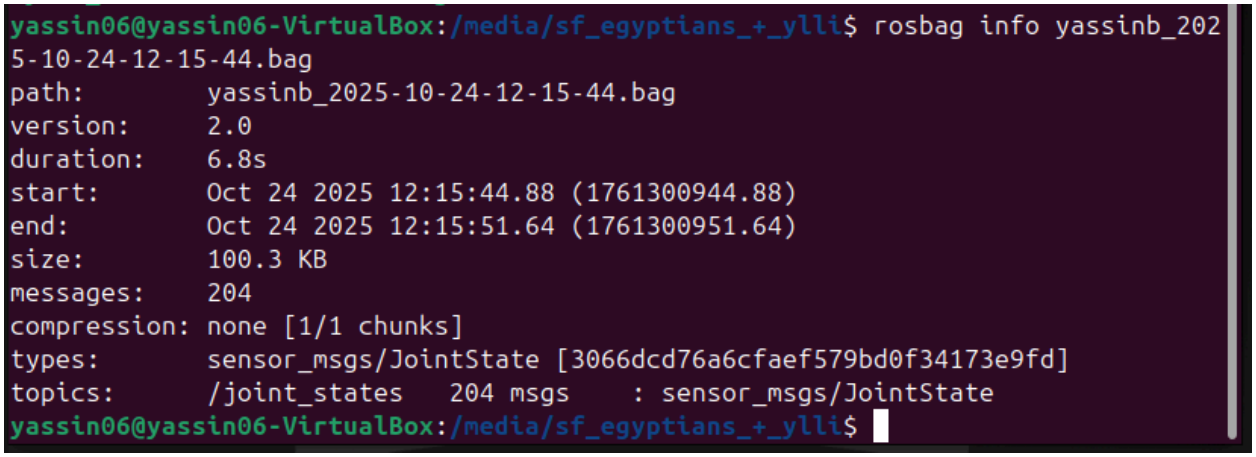


The Panda robot's structure and kinematics are defined in its URDF file, which is automatically loaded into the ROS parameter /robot_description when launching the robot. The file describes all the robot's links and movable joints, including their motion limits such as effort, range, and velocity. In the URDF, one of the joints, for example, panda_joint1 is defined as a revolute joint with limits of:

- Lower: –2.8973 rad
- Upper: +2.8973 rad
- Effort: 87 N·m
- Velocity: 2.175 rad/s

These parameters define how far and how fast the joint can rotate, ensuring that the Panda robot moves safely and smoothly during both simulation and real-world operation.

Figure 19 Verification of recorded /joint_states data using rosbag info:



For the Panda robot, we recorded a short rosbag of the /joint_states topic while moving the arm in freedrive mode to write the first letter of our name in the air. The motion was shown in RViz using the TFTrajectory display. Figure 19 shows that the file yassinb_2025-10-24-12-15-44.bag contains 204 joint-state messages over 6.8 seconds, confirming that the recording worked correctly.

Task 3:

Each of the three robotic systems UR5e, Kinova, and Panda, has unique advantages and disadvantages depending on the purpose they are used for.

- UR5e:
  We think the UR5e is the easiest to use for beginners because it's very reliable and simple to set up. It helped us understand the basics of robot control, kinematics, and ROS topics without too much complexity. However, we also noticed that it lacks advanced sensing features like force feedback, which makes it less suitable for delicate or precise manipulation tasks..

- Kinova:
  From our experience, the Kinova arm felt more advanced thanks to its torque sensors and integrated gripper. It's great for research projects and for testing human robot interaction concepts. The only downside we observed was that it took more time to initialize and configure properly compared to the UR5e, but once it was running, it worked very smoothly.

- Panda:
  The Panda robot gave us the impression of being the most precise and well-controlled system. Its motion was extremely smooth, and the safety features were noticeable when we tried different movements. We think it's perfect for fine manipulation or research

tasks that need high accuracy, although it can feel a bit restrictive due to the built-in safety limits. It's also more expensive and complex, which might not make it ideal for basic experiments.

Contributions:

The lab tasks during physical lab: all participated equally to finish the tasks.

Lab report: Yassin Baher Youssef